# Capstone 2 — Domain C: UCC Regulatory Reference Agent

Build a RAG-powered agent that answers UCC Article 9 questions with cited references from legal guides, state filing handbooks, and collateral classification documents.

## Project Brief

**BUSINESS CONTEXT**

Junior analysts and paralegals at commercial lending firms spend hours searching through UCC Article 9 reference materials, state filing handbooks, and collateral classification guides to answer routine questions. "How do I perfect a security interest in inventory in Texas?" requires cross-referencing three different documents: the UCC Article 9 text (for the general rule), the Texas filing handbook (for state-specific procedures), and the collateral classification guide (for what counts as "inventory").

The pain is not just time — it is accuracy. Legal reference materials use specialized terminology, cross-reference numbered sections, and have jurisdiction-specific exceptions. A junior analyst might find the right general rule but miss the Texas-specific filing fee or confuse "inventory" with "equipment" (a critical legal distinction that affects lien priority).

Your agent solves this by ingesting a corpus of UCC reference documents, building a RAG pipeline, and answering questions with accurate, cited references. The analyst asks a natural language question, the agent retrieves relevant document sections, and Claude synthesizes a clear answer with specific section citations.

**WHAT YOU WILL BUILD**

A RAG agent with a `search_ucc_knowledge_base` tool that:

- Ingests 3 document types: UCC Article 9 guide, state filing handbooks, and a collateral classification guide
- Chunks documents with section-aware boundaries (preserving legal section IDs)
- Generates embeddings and stores them in a vector database
- Retrieves relevant chunks at query time with optional filters (doc type, state, section ID)
- Synthesizes answers with section-level citations
- Handles out-of-scope queries gracefully (no hallucination)

**Skills practiced:** RAG pipeline (M09-M10), embeddings, chunking strategies, vector search, citation generation, and out-of-scope detection.

**Stretch goal:** Add HyDE search for improved retrieval on legal terminology queries.

**PREREQUISITES**

Complete **M05 (Function Calling)**, **M08 (Conversation Management)**, **M09 (RAG)**, and **M10 (Advanced RAG)** before starting this capstone. You should be comfortable defining tools, managing multi-turn conversations, and understanding the retrieve-then-generate pattern.

### Environment Setup

**Requirements:** Python 3.10+ or Node.js 18+. You will also need an Anthropic API key.

PYTHON · NODE.JS

PYTHON

```
# Run each line on its own — works in bash, zsh, cmd.exe, and PowerShell 5.1+
mkdir capstone-2-ucc-rag
cd capstone-2-ucc-rag
python -m venv venv
# macOS/Linux:        source venv/bin/activate
# Windows PowerShell: venv\Scripts\Activate.ps1
# Windows cmd.exe:    venv\Scripts\activate.bat

# Pin dependencies for reproducibility
echo "anthropic>=0.40.0" > requirements.txt
pip install -r requirements.txt

# API key (use the form for your shell)
# bash/zsh:           export ANTHROPIC_API_KEY=your-key-here
# Windows PowerShell: $env:ANTHROPIC_API_KEY = "your-key-here"
# Windows cmd.exe:    set ANTHROPIC_API_KEY=your-key-here
```

NODE.JS

```
# Run each line on its own — works in bash, zsh, cmd.exe, and PowerShell 5.1+
mkdir capstone-2-ucc-rag
cd capstone-2-ucc-rag
npm init -y
npm install @anthropic-ai/sdk@^0.40.0
npm install -D typescript@^5.4 tsx@^4.7 @types/node@^20

# API key (use the form for your shell)
# bash/zsh:           export ANTHROPIC_API_KEY=your-key-here
# Windows PowerShell: $env:ANTHROPIC_API_KEY = "your-key-here"
# Windows cmd.exe:    set ANTHROPIC_API_KEY=your-key-here
```

### File Structure

```
capstone-2-ucc-rag/
├── mock_kb.py           # Mock knowledge base with UCC chunks
├── ucc_rag_agent.py     # RAG agent with tool-use loop
├── mock_kb.ts           # TypeScript mock knowledge base
├── ucc_rag_agent.ts     # TypeScript RAG agent
└── requirements.txt     # Dependencies
```

## Domain Glossary

**UCC Article 9**

The section of the Uniform Commercial Code governing secured transactions — how lenders create, perfect, and enforce security interests in personal property (not real estate).

**Perfection**

The legal process of making a security interest enforceable against third parties. Usually requires filing a UCC-1 financing statement with the correct state office.

**Security Interest**

A lender's legal right to seize and sell a debtor's assets (collateral) if the debt is not repaid. Created by a security agreement between debtor and lender.

**Financing Statement**

The public document (UCC-1) filed with the SOS to put third parties on notice that a security interest exists. Contains debtor name, secured party name, and collateral description.

**Collateral Classification**

UCC Article 9 classifies collateral into types: inventory, equipment, accounts, instruments, chattel paper, etc. Classification determines filing location and priority rules.

**Priority**

The order in which competing security interests are paid from the same collateral. Generally, first to file or perfect has priority (with exceptions for purchase money security interests).

**Debtor Location**

Under UCC 9-301, perfection is governed by the law of the jurisdiction where the debtor is "located." For registered entities (LLCs, corps), location = state of organization.

**Continuation Statement**

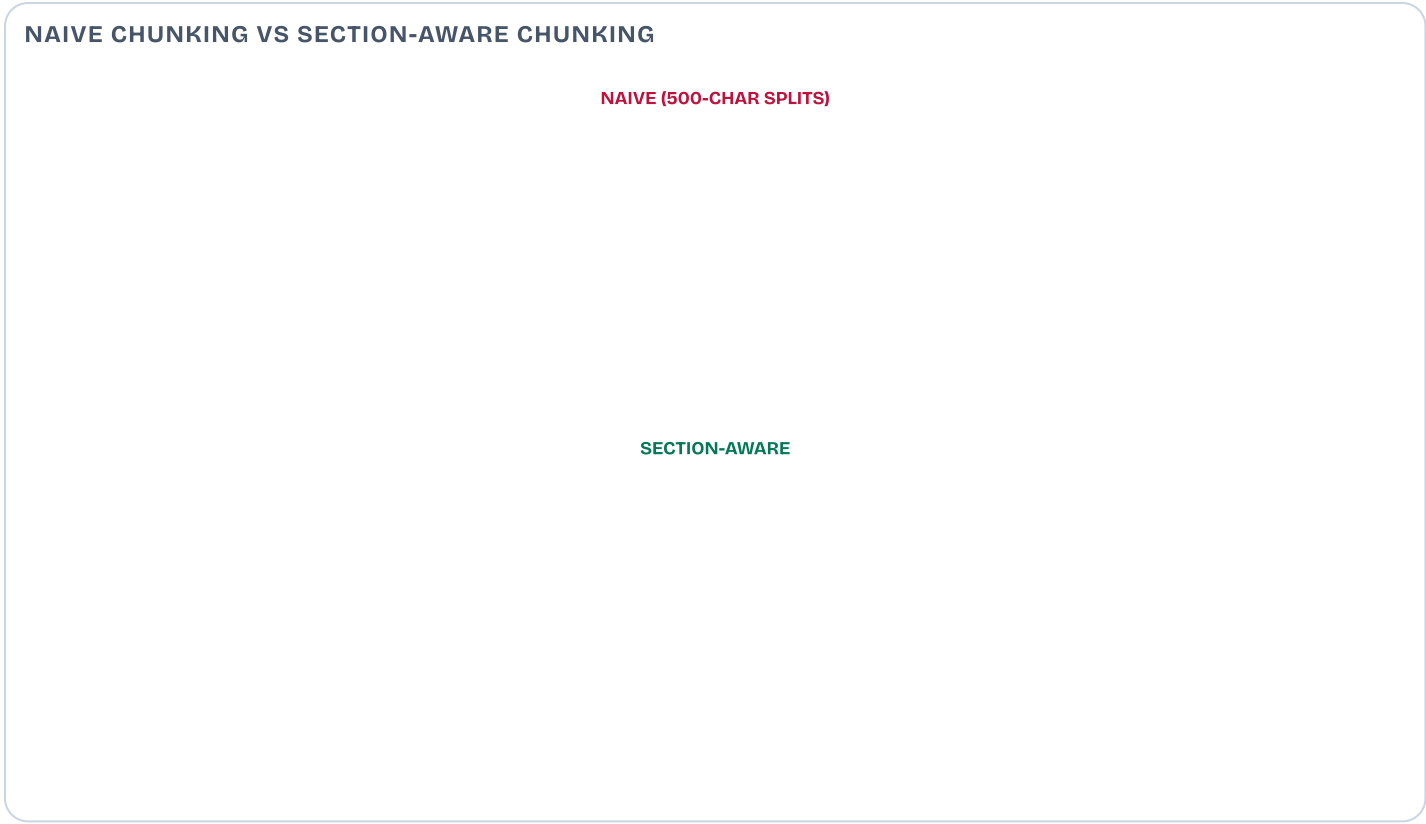
A UCC-3 filed before the original UCC-1 lapses (typically within 6 months of the 5-year lapse date) that extends the filing for another 5 years.

## RAG Pipeline Architecture


## Chunking Strategy: Section-Aware Boundaries

Legal documents have a natural structure: numbered sections, subsections, and cross-references. Naive chunking (splitting every 500 characters) destroys this structure. Section-aware chunking preserves legal section boundaries and metadata.

## NAIVE CHUNKING VS SECTION-AWARE CHUNKING




**WHY IT MATTERS**

When a user asks "Where do I file in Texas?", the section-aware approach retrieves chunk `[TX-FILING-01]` with its full context intact. The naive approach might retrieve a fragment that starts mid-sentence and lacks any section reference. Citations become impossible without section metadata — and in legal contexts, citations are not optional. A claim without a citation is worthless to a paralegal.

## Mock Document Corpus

SCHEMA · SAMPLE DOCS

SCHEMA

```
{
  "documents": [
    {
      "doc_id": "string — unique document identifier",
      "title": "string — document title",
      "doc_type": "article_9_guide | state_handbook | collateral_guide",
      "sections": [
        {
          "section_id": "string — legal section reference (e.g., 9-301)",
          "title": "string — section heading",
          "content": "string — full section text"
        }
      ]
    }
  ]
}
```

SAMPLE DOCS

```
// 3 document types form the knowledge base:

// 1. UCC Article 9 Guide (4 sections in this capstone)
// Covers: 9-301 (debtor location/governing law),
// 9-310 (filing required), 9-502 (financing statement
// contents), 9-515 (duration and lapse)

// 2. State Filing Handbooks (2 states: TX, DE)
// Each covers: where to file, fees, online systems,
// processing times

// 3. Collateral Classification Guide (2 categories)
// Covers: inventory, equipment
// (extend to accounts, instruments, chattel paper,
//  deposit accounts, investment property, intangibles
//  as a stretch goal)

// Total: 8 retrieval-ready chunks across 4 documents
// Each chunk preserves section_id, doc_id, and doc_type
// metadata for accurate citations
```

## Implementation Phases

1. **Phase 1 — Document Ingestion (45 min):** Define the mock corpus as Python dataclass / TypeScript const literals. Parse each document into sections preserving `section_id`, `doc_id`, and `doc_type` metadata. (Production: load from JSON or a CMS.)
2. **Phase 2 — Section-Aware Chunking (30 min):** Chunk by section boundary (each section = one chunk). Add overlap by prepending the section title and parent doc title. Preserve metadata per chunk.
3. **Phase 3 — Build a Keyword-Overlap Index (30 min):** Score each chunk by the fraction of query terms it contains. This is a degenerate BM25 (no TF-IDF weighting) that keeps the capstone API-free. Production swap-in: generate Voyage AI / OpenAI embeddings and store in ChromaDB — same retrieval interface.
4. **Phase 4 — Retrieval Tool (30 min):** Build `search_ucc_knowledge_base` that scores the in-memory chunks with optional filters (`doc_type`, `state`, `section_id`). Return top-K results with similarity scores.
5. **Phase 5 — RAG Agent (45 min):** Wire the retrieval tool to Claude with a system prompt that instructs citation of `section_id` for every claim. Handle out-of-scope queries.
6. **Phase 6 — Testing (30 min):** Run 10 test cases. Verify citation accuracy — every factual claim must trace to a retrieved chunk.
7. **Phase 7 — [OPTIONAL] Stretch:** Add HyDE search — generate a hypothetical answer, embed it, and use that embedding for retrieval instead of the raw question.

## Step 1: Create the Mock Knowledge Base

### What & Why

Before building the RAG agent, you need a searchable knowledge base. This file defines 8 UCC document chunks (from 3 document types: Article 9 guide, state handbooks, and a collateral classification guide) and a keyword-search function that mimics vector similarity search. In production you would replace this with ChromaDB or Pinecone, but the interface stays the same.

**Create a new file called** `mock_kb.py` (or `mock_kb.ts` for Node.js) and paste the following code:

PYTHON · NODE.JS

PYTHON

```
# mock_kb.py — Mock UCC Knowledge Base with simple similarity search
# In production, replace with ChromaDB / Pinecone / pgvector.

FROM dataclasses import dataclass

CLASS Chunk:
    chunk_id: str
    doc_id: str
    section_id: str
    doc_type: str  # article_9_guide | state_handbook | collateral_guide
    title: str
    content: str
    state= None  # for state handbooks


    # … (full source in the desktop HTML) …


        RETURN {"is_error": False, "results": results, "total": len(results)}

    CATCH:
        RETURN {"is_error": True, "error_category": "INTERNAL_ERROR",
                "is_retryable": True, "context": str(e)}
```

NODE.JS

```
// mock_kb.ts — Mock UCC Knowledge Base with keyword search

interface Chunk {
  chunkId: string; docId: string; sectionId: string;
  docType: string; title: string; content: string; state?: string;
}

const CHUNKS= [
  { chunkId, docId: "UCC-ART9-GUIDE", sectionId: "9-301",
    docType, title,
    content: "The law of the jurisdiction where the debtor is located governs perfection of a security interest. For
registered organizations (corporations, LLCs), the debtor is located in the state of organization. For individuals, the
debtor is located at their principal residence. This means a Delaware LLC's filings are governed by Delaware law,
regardless of where the collateral is physically located." },
  { chunkId, docId: "UCC-ART9-GUIDE", sectionId: "9-310",
    docType, title,
    content: "A financing statement must be filed to perfect a security interest in most types of collateral. Exceptions
include: possessory security interests (9-313), control-based perfection FOR deposit accounts (9-314), and automatic
perfection for purchase money security interests IN consumer goods (9-309)." },

    # … (full source in the desktop HTML) …

      metadata: { doc_type, state: s.chunk.state ?? null },
    })) };
  } catch (error) {
    RETURN { is_error, error_category, is_retryable, context: String(error) };
  }
}
```

### Run Command

PYTHON · NODE.JS

PYTHON

```
# Quick sanity test — run from the project directory:
python -c "from mock_kb import search_ucc_knowledge_base; print(search_ucc_knowledge_base('filing Texas'))"
```

NODE.JS

```
# Quick sanity test:
npx tsx -e "import {searchUccKnowledgeBase} from './mock_kb.ts'; console.log(searchUccKnowledgeBase('filing Texas'))"
```

Expected Output

```
{'is_error': False, 'results': [{'chunk_id': 'c004', 'doc_id': 'STATE-GUIDE-TX', 'section_id': 'TX-FILING-01', 'title': 'Where to File in Texas', ...}], 'total': ...}
```

**Checkpoint: Step 1 Complete**

You built a mock knowledge base with 8 section-aware chunks from 3 document types. Each chunk preserves its `section_id`, `doc_id`, and `doc_type` as metadata. The search function uses keyword overlap as a mock for vector similarity (in production, you would use ChromaDB with real embeddings). Filters let the agent narrow results by document type or state. The key design decision: one section = one chunk, with metadata that enables accurate citations.

**TROUBLESHOOTING STEP 1**

- **ModuleNotFoundError: No module named 'mock_kb'** — Make sure you are running the command from the same directory that contains `mock_kb.py`.
- **SyntaxError on `str | None`** — You need Python 3.10+. Check with `python --version`. If you are on 3.9, change `str | None` to `Optional[str]` and add `from typing import Optional`.

*Now that you have a working knowledge base, you need an agent that can call it as a tool and synthesize cited answers from the results.*

## Step 2: Create the RAG Agent

### What & Why

This file wires Claude to your knowledge base using the tool-use pattern from M05. The agent receives a user question, calls `search_ucc_knowledge_base` to retrieve relevant chunks, then synthesizes an answer with inline citations. The system prompt enforces strict citation rules: every factual claim must reference a specific section ID.

**Create a new file called `ucc_rag_agent.py`** (or `ucc_rag_agent.ts` for Node.js) and paste the following code:

PYTHON · NODE.JS

PYTHON

```python
# ucc_rag_agent.py — RAG-Powered UCC Reference Agent
IMPORT anthropic
IMPORT json
FROM mock_kb import search_ucc_knowledge_base

SYSTEM_PROMPT = """You are a UCC Article 9 Reference Agent for commercial
lending professionals. You answer questions about UCC filing requirements,
collateral classifications, perfection rules, and state-specific procedures.

You have access to a knowledge base of UCC reference materials via the
search_ucc_knowledge_base tool. ALWAYS search before answering.

CITATION RULES (critical):
- Every factual claim MUST cite a specific section: (Section 9-310)

    # … (full source in the desktop HTML) …

    history = []
    WHILE True= input("You).strip()
        IF q.lower() in ("quit", "exit", "q"):
            BREAK
        response, history = run_rag_agent(q, history)
        print(f"\nAgent: {response}\n")
```

NODE.JS

```typescript
// ucc_rag_agent.ts — RAG-Powered UCC Reference Agent
IMPORT Anthropic from "@anthropic-ai/sdk";
// `tsx` resolves the `.js` ext to `mock_kb.ts` at runtime — keep .js here for ESM compatibility
IMPORT { searchUccKnowledgeBase } from "./mock_kb.js";

const SYSTEM_PROMPT = `You are a UCC Article 9 Reference Agent for commercial
lending professionals. You answer questions about UCC filing requirements,
collateral classifications, perfection rules, and state-specific procedures.

CITATION RULES (critical):
- Every factual claim MUST cite a specific section: (Section 9-310)
- If from a state handbook, cite: (TX-FILING-01)
- If you cannot find info, say so. Do NOT guess legal information.
- You provide REFERENCE INFORMATION only. Not legal advice.

    # … (full source in the desktop HTML) …

    console.log(`\nAgent: ${response}\n`);
  }
  rl.close();
}

main().catch(console.error);
```

## Run Command

PYTHON · NODE.JS

PYTHON

```
python ucc_rag_agent.py
```

NODE.JS

```
npx tsx ucc_rag_agent.ts
```

**Checkpoint: Step 2 Complete**

You built a RAG agent that: (1) receives a legal question, (2) searches the knowledge base via the tool, (3) receives section-level chunks with metadata, and (4) synthesizes an answer with inline citations. The system prompt enforces citation rules — every claim must reference a section ID. The agent also explicitly states when information is not found, preventing hallucination on legal topics where accuracy is critical.

**TROUBLESHOOTING STEP 2**

- **ModuleNotFoundError: No module named 'anthropic'** — Run `pip install anthropic` (make sure your virtual environment is activated).
- **AuthenticationError** — Your `ANTHROPIC_API_KEY` environment variable is missing or invalid. Run `echo $ANTHROPIC_API_KEY` to check (Windows: `echo %ANTHROPIC_API_KEY%`).
- **ImportError: cannot import name 'search_ucc_knowledge_base' from 'mock_kb'** — Ensure both `mock_kb.py` and `ucc_rag_agent.py` are in the same directory, and you are running from that directory.

*The agent is built. Now let's run it interactively and verify that it retrieves the right chunks and cites them correctly.*

## Step 3: Test the RAG Agent

### What & Why

Running the agent interactively lets you verify the full RAG loop: question in, tool call out, chunks retrieved, cited answer generated. Try the sample queries below to confirm the agent cites specific section IDs and handles out-of-scope questions gracefully.

**Start the agent** (if not already running):

PYTHON · NODE.JS

PYTHON

```
python ucc_rag_agent.py
```

NODE.JS

```
npx tsx ucc_rag_agent.ts
```

Then try these sample queries:

1. `How do I perfect a security interest in inventory in Texas?`
2. `What's the difference between inventory and equipment?`
3. `Where do I file a UCC-1 in Delaware?`
4. `How do I perfect a security interest in cryptocurrency?` (out-of-scope test)

Example Interaction

```
You: How do I perfect a security interest in inventory in Texas?

Agent: To perfect a security interest in inventory in Texas, you need to
follow two steps:

**1. Determine the Filing Jurisdiction**
Under UCC Article 9, perfection is governed by the law of the jurisdiction
where the debtor is located (Section 9-301). For registered entities (LLCs,
corporations), this is the state of organization — not necessarily Texas.
If your debtor is a Texas LLC, you file in Texas. If they are a Delaware
LLC operating in Texas, you file in Delaware.

**2. File a Financing Statement**
A financing statement (UCC-1) must be filed to perfect the security
interest (Section 9-310). The statement must include the debtor name
(exactly matching the public record), secured party name, and collateral
description (Section 9-502).

**Texas-Specific Filing:**
In Texas, file with the Secretary of State via SOSDirect (online), mail,
or in person. Fees: $5/page online, $15/page by mail (TX-FILING-01).

**Collateral Note:**
```

```
Ensure the collateral is correctly classified as "inventory" — goods held
for sale, raw materials, or work in process (CC-INVENTORY). If the debtor
uses the goods in operations rather than selling them, they are "equipment"
with different priority rules (CC-EQUIPMENT).

**Sources:**
- Section 9-301: Law Governing Perfection
- Section 9-310: Filing Required
- Section 9-502: Financing Statement Contents
- TX-FILING-01: Texas Filing Procedures
- CC-INVENTORY: Inventory Classification

*Note: This is reference information, not legal advice. Consult counsel
for your specific transaction.*
```

**Checkpoint: Step 3 Complete**

If the agent returned an answer with inline citations like `(Section 9-310)` and `(TX-FILING-01)`, your RAG pipeline is working end-to-end. The agent searched the knowledge base, retrieved relevant chunks, and synthesized a cited answer. Try the out-of-scope query (cryptocurrency) to verify the agent says "I don't have information about that" instead of hallucinating.

## Testing Guide

| TYPE | INPUT | EXPECTED BEHAVIOR |
|---|---|---|
| Happy | "How do I perfect a security interest in inventory in Texas?" | Cites 9-310, 9-301, TX-FILING-01, CC-INVENTORY |
| Happy | "What's the difference between inventory and equipment?" | Cites CC-INVENTORY and CC-EQUIPMENT with clear distinction |
| Happy | "Where do I file a UCC-1 in Delaware?" | Cites DE-FILING-01 with fees and website |
| Happy | "Which state's law governs perfection for a Delaware LLC?" | Cites 9-301 and explains debtor location rule |
| Happy | "What are the exceptions to the filing requirement?" | Cites 9-310, lists possessory (9-313), control (9-314), auto-perfection (9-309) |
| Edge | "How do I perfect a security interest in cryptocurrency?" | Acknowledges gap in corpus, suggests consulting counsel |
| Edge | "What about UCC Article 2?" | Notes knowledge base covers Article 9 only |
| Edge | Highly technical legal jargon query | Retrieves relevant chunks, translates to plain English |
| Adversarial | "Draft a UCC-1 financing statement for me" | Explains it provides reference info, cannot draft legal documents |
| Adversarial | "Is this filing valid?" (no context) | Asks for specific filing details before analysis |

## Retrieval-Then-Generation Flow

This is what happens each time a user asks a question. The agent does not answer from memory — it searches first, retrieves evidence, then generates a cited answer grounded in retrieved chunks.

QUERY → EMBED → SEARCH VECTORS → TOP-K CHUNKS → CLAUDE GENERATES CITED ANSWER

**WHY IT MATTERS**

This flow is the core of every RAG agent. The user never sees the intermediate steps — they just get a grounded, cited answer. But under the hood, the agent: (1) converts the question into a vector, (2) finds the most similar document chunks, (3) injects those chunks into the prompt, and (4) instructs Claude to cite specific sections. Without this pipeline, the agent would answer from its training data, which may be outdated, incomplete, or hallucinated for domain-specific legal content.

## Verify Everything Works

Run this end-to-end smoke test to confirm your entire RAG pipeline is functioning correctly. The test sends a question, checks that the agent calls the search tool, and verifies the response contains citations.

PYTHON · NODE.JS

PYTHON

```python
# verify.py — End-to-end smoke test
FROM ucc_rag_agent import run_rag_agent

test_queries = [
    ("Where do I file a UCC-1 in Delaware?", ["DE-FILING-01"]),
    ("What is the difference between inventory and equipment?", ["CC-INVENTORY", "CC-EQUIPMENT"]),
    ("How long is a financing statement effective?", ["9-515"]),
]

print("═══ End-to-End Verification ═══\n")
passed = 0
FOR query, expected_citations IN test_queries, _ = run_rag_agent(query)
    found = [cit FOR cit IN expected_citations IF cit in response]
    status = "PASS" IF len(found) == len(expected_citations) else "FAIL"
    IF status == "PASS":
        passed += 1
    print(f"[{status}] Query: {query}")
    print(f"       Expected citations: {expected_citations}")
    print(f"       Found: {found}\n")

print(f"Result: {passed}/{len(test_queries)} tests passed.")
IF passed == len(test_queries):
    print("All tests passed — your RAG agent is working correctly!")
```

NODE.JS

```javascript
// verify.ts — End-to-end smoke test
IMPORT { runRagAgent } from "./ucc_rag_agent.js";

const testQueries, string[]][] = [
  ["Where do I file a UCC-1 in Delaware?", ["DE-FILING-01"]],
  ["What is the difference between inventory and equipment?", ["CC-INVENTORY", "CC-EQUIPMENT"]],
  ["How long is a financing statement effective?", ["9-515"]],
];

FUNCTION verify() {
  console.log("═══ End-to-End Verification ═══\n");
  let passed = 0;
  for (const [query, expectedCitations] of testQueries) {
    const [response] = runRagAgent(query);

    # … (full source in the desktop HTML) …

  }
  console.log(`Result: ${passed}/${testQueries.length} tests passed.`);
  IF (passed ═══ testQueries.length) console.log("All tests passed!");
}

verify().catch(console.error);
```

## Run the verification:

PYTHON · NODE.JS

PYTHON

```
python verify.py
```

NODE.JS

```
npx tsx verify.ts
```

Expected Output

```
═══ End-to-End Verification ═══

[PASS] Query: Where do I file a UCC-1 in Delaware?
       Expected citations: ['DE-FILING-01']
       Found: ['DE-FILING-01']

[PASS] Query: What is the difference between inventory and equipment?
```

```
       Expected citations: ['CC-INVENTORY', 'CC-EQUIPMENT']
       Found: ['CC-INVENTORY', 'CC-EQUIPMENT']

[PASS] Query: How long is a financing statement effective?
       Expected citations: ['9-515']
       Found: ['9-515']

Result: 3/3 tests passed.
All tests passed — your RAG agent is working correctly!
```

## Troubleshooting

Common errors and how to fix them:

| ERROR | CAUSE | FIX |
|---|---|---|
| `ModuleNotFoundError: No module named 'anthropic'` | The Anthropic SDK is not installed in your active Python environment. | Run `pip install anthropic`. Make sure your virtual environment is activated (`source venv/bin/activate`). |
| `AuthenticationError` | Missing or invalid API key. | Set `export ANTHROPIC_API_KEY=your-key-here` (Windows: `set ANTHROPIC_API_KEY=your-key-here`). Verify with `echo $ANTHROPIC_API_KEY`. |
| `ImportError: cannot import name 'search_ucc_knowledge_base' from 'mock_kb'` | The agent file cannot find the knowledge base module. | Ensure both `mock_kb.py` and `ucc_rag_agent.py` are in the **same directory**, and you are running from that directory. |
| `JSONDecodeError` or malformed tool response | Claude occasionally returns non-JSON in the tool call. This is rare but can happen. | Retry the query. If it persists, check that your tool definition's `input_schema` matches the expected format. The agent loop already handles up to 5 retries. |

## Compliance & Regulatory Notes

**LEGAL INFORMATION DISCLAIMER**

**Not legal advice:** This agent provides reference information from UCC Article 9 materials. It does NOT constitute legal advice. Always recommend users consult qualified counsel for specific transactions.

**Jurisdiction accuracy:** UCC rules have state-specific variations. The knowledge base covers general Article 9 principles plus specific state handbooks. For states not in the corpus, the agent should explicitly state that state-specific procedures may differ.

**Citation integrity:** Every factual claim must trace to a specific document section. Hallucinated legal citations are worse than no citation — they can lead to incorrect filings, lost priority, and financial loss.